

Expressions régulières en R

Aide-mémoire

Classes de Caractères

<code>[[digit:]]</code> ou <code>\\d</code>	Chiffres décimaux; [0-9]
<code>\\D</code>	Tout sauf chiffres décimaux; [^0-9]
<code>[:lower:]</code>	Lettres minuscules; [a-z]
<code>[:upper:]</code>	Lettres majuscules; [A-Z]
<code>[:alpha:]</code>	Caractères alphabétiques; [A-z]
<code>[:alnum:]</code>	Caractères alphanumériques; [A-z0-9]
<code>\\w</code>	Caractères alphanumériques ou tirets; [A-z0-9_]
<code>\\W</code>	Tout sauf caractères alphanumériques ou tirets
<code>[:xdigit:]</code> ou <code>\\x</code>	Chiffres hexadécimaux; [0-9A-Fa-f]
<code>[:blank:]</code>	Espace et tabulation
<code>[:space:]</code> ou <code>\\s</code>	Caractères d'espacement par ex. espace, tabulation, retour chariot, etc
<code>\\S</code>	Tout sauf caractères d'espacement; [^[:space:]]
<code>[:punct:]</code>	Signes de ponctuation; !"#\$%&'()*+,-./:;<=>?@[\\]^_`{ }~
<code>[:graph:]</code>	Caractères d'imprimerie; [[:alnum:][:punct:]]
<code>[:print:]</code>	Caractères imprimables; [[:alnum:][:punct:][:\\s]]
<code>[:cntrl:]</code> ou <code>\\c</code>	Caractères de contrôle; \\n, \\r etc.

Métacaractères Spéciaux

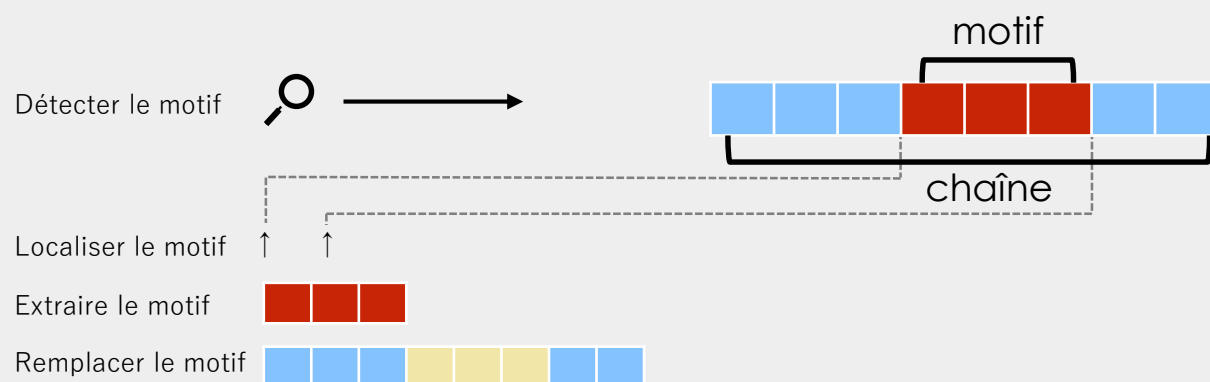
<code>\\n</code>	Nouvelle ligne
<code>\\r</code>	Retour chariot
<code>\\t</code>	Tabulation
<code>\\v</code>	Tabulation verticale
<code>\\f</code>	Saut de page

Assertions et Conditionnels*

<code>(?=)</code>	Assertion avant (PERL = TRUE), par ex. <code>(?=yx)</code> : position suivie par 'xy'
<code>(?!)</code>	Assertion avant négative (PERL = TRUE); position NON suivie par un motif
<code>(?<=)</code>	Assertion arrière (PERL = TRUE), par ex. <code>(?<=yx)</code> : position suit 'xy'
<code>(?<!)</code>	Assertion arrière négative (PERL = TRUE); position NE suit PAS un motif
<code>?(if)then</code>	If-then-condition (PERL = TRUE); utilise assertion avant, caractère optionel, etc dans la clause if
<code>?(if)then else</code>	If-then-else-condition (PERL = TRUE)

*see, e.g. <http://www.regular-expressions.info/lookaround.html>
<http://www.regular-expressions.info/conditional.html>

Fonctions pour l'Appariement de Motifs



```
> chaîne <- c("Hiphopopotamus", "Rhymenoceros", "time for bottomless lyrics")
> motif <- "t.m"
```

Détecter des motifs

```
grep(motif, chaîne)
[1] 1 3

grep(motif, chaîne, value = TRUE)
[1] "Hiphopopotamus"
[2] "time for bottomless lyrics"

grepl(motif, chaîne)
[1] TRUE FALSE TRUE

stringr::str_detect(chaîne, motif)
[1] TRUE FALSE TRUE
```

Localiser des motifs

```
regexpr(motif, chaîne)
Début et longueur de la première correspondance

gregexpr(motif, chaîne)
Début et longueur de toutes les correspondances

stringr::str_locate(chaîne, motif)
Début et longueur de la première correspondance

stringr::str_locate_all(chaîne, motif)
Début et longueur de toutes les correspondances
```

Séparer une chaîne en utilisant un motif

```
strsplit(chaîne, motif) ou stringr::str_split(chaîne, motif)
```

Extraire des motifs

```
regmatches(chaîne, regexpr(motif, chaîne))
extrait la première correspondance [1] "tam" "tim"

regmatches(chaîne, gregexpr(motif, chaîne))
extrait toutes les correspondances, retourne une liste
[[1]] "tam" [[2]] character(0) [[3]] "tim" "tom"

stringr::str_extract(chaîne, motif)
extrait la première correspondance [1] "tam" NA "tim"

stringr::str_extract_all(chaîne, motif)
extrait toutes les correspondances, retourne une liste

stringr::str_extract_all(chaîne, motif, simplify = TRUE)
extrait toutes les correspondances, retourne une matrice

stringr::str_match(chaîne, motif)
extrait le premier groupe correspondant

stringr::str_match_all(chaîne, motif)
extrait tous les groupes correspondants
```

Remplacer des motifs

```
sub(motif, remplacement, chaîne)
remplacer la première correspondance

gsub(motif, remplacement, chaîne)
remplacer toutes les correspondances

stringr::str_replace( chaîne, motif, remplacement)
remplacer la première correspondance

stringr::str_replace_all(chaîne, motif, remplacement)
remplacer toutes les correspondances
```

Classes et Groupes de Caractères

<code>.</code>	N'importe quel caractère sauf \\n
<code> </code>	Ou, e.g. <code>(a b)</code>
<code>[...]</code>	Lister les caractères autorisés, ex. <code>[abc]</code>
<code>[a-z]</code>	Spécifier la plage des caractères
<code>^[...]</code>	Lister les caractères exclus
<code>(...)</code>	Grouper, permet d'y faire référence avec \\N où N est un entier

Mode général

Par défaut, R utilise les expressions régulières étendues (POSIX). Vous pouvez utiliser PCRE en R-base en spécifiant `PERL = TRUE` ou en utilisant `perl(motif)` pour `stringr`

Les recherches exactes peuvent être faites en utilisant `fixed = TRUE` en R-base ou en utilisant `fixed(motif)` pour `stringr`. Elles deviennent insensibles à la casse en R-base en spécifiant `ignore.case = TRUE` et `regex(motif, ignore_case = TRUE)` pour `stringr`

Ancres

<code>^</code>	Début de la chaîne de caractère
<code>\$</code>	Fin de la chaîne de caractère
<code>\\b</code>	Chaîne vide à la limite d'un mot
<code>\\B</code>	Pas à la limite du mot
<code>\\<</code>	Début du mot
<code>\\></code>	Fin du mot

Caractères d'échappement

Les métacaractères (`.` `*` `+` etc.) doivent être échappés pour être utilisés littéralement.

Pour échapper un caractère, on place `\\` avant le caractère ou on l'encadre par `\\Q...\\E`.

Conversions de casses

Les expressions régulières peuvent être rendues insensibles à la casse avec `(?i)`. Il est ensuite possible d'y faire référence et convertir la chaîne en minuscule ou majuscule avec respectivement `\\L` ou `\\U` (par ex. `\\L\\1`). Ça demande au préalable `PERL = TRUE`.

Quantifieurs

<code>*</code>	Correspond au moins zéro fois
<code>+</code>	Correspond au moins une fois
<code>?</code>	Correspond au plus une fois
<code>{n}</code>	Correspond exactement n fois
<code>{n,}</code>	Correspond au moins n fois
<code>{n,m}</code>	Correspond entre n et m fois

Appariement gourmand

Par défaut, l'asterique `*` est gourmand, c.-à-d. il apparie toujours la chaîne la plus longue possible. Il peut être rendu paresseux avec `?`, c.-à-d. `*?`. Pour désactiver Le mode gourmand il faut utiliser `(?U)`. Cela active une syntaxe qui rend `(?U)a*` paresseux et `(?U)a*?` gourmand.

Note

Les expressions régulières peuvent être créées plus convivialement en utilisant des packages R comme `rex` ou `rebus`.